

Yocto 1

Core Concepts

BitBake Language & Recipe Crafting

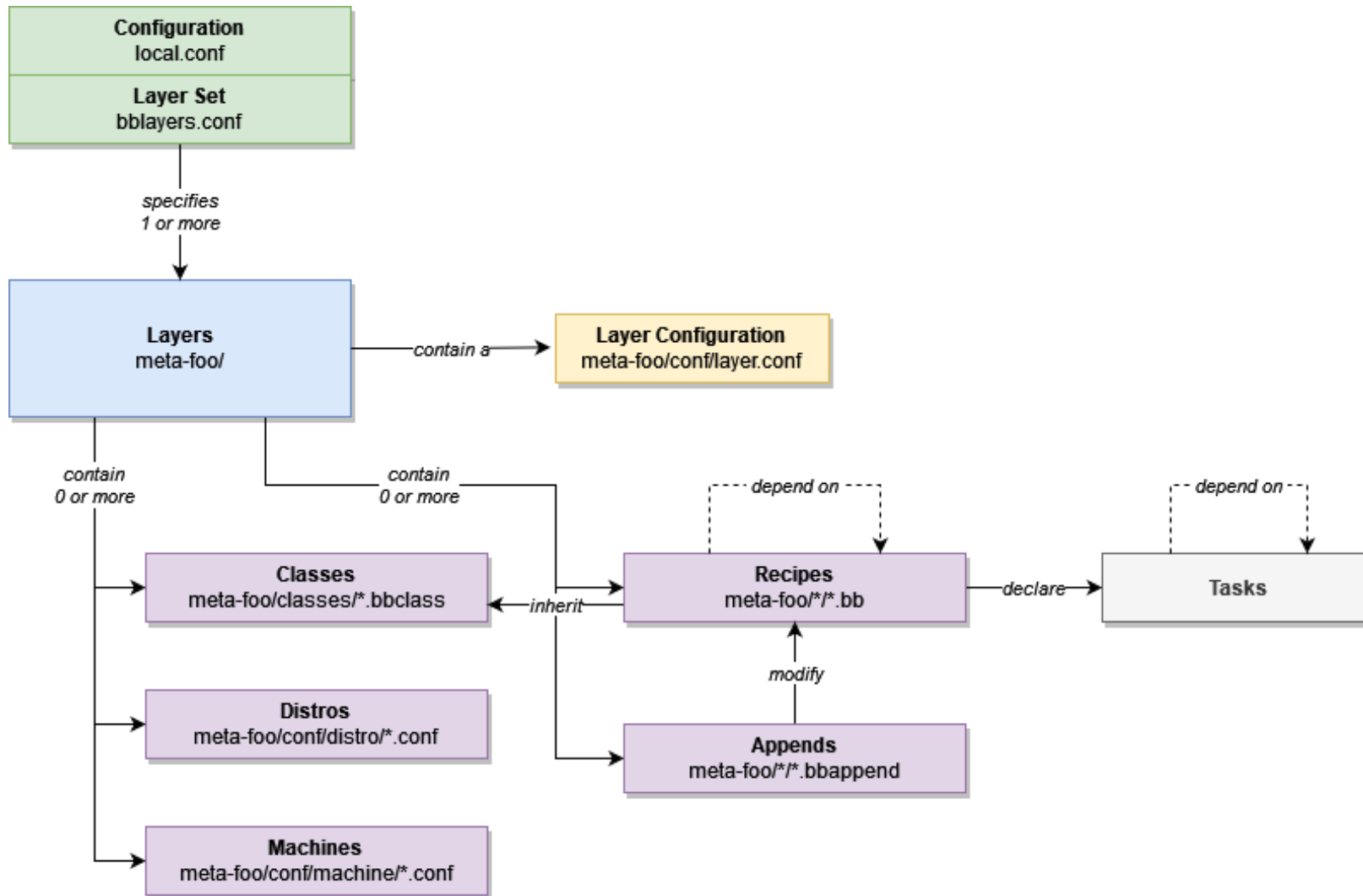
Tools & Troubleshooting

Yocto 1 – OpenEmbedded, or Yocto, or BitBake, or Poky?

- [OpenEmbedded](#) (2003)
 - Evolved from PDA firmware projects in the 90s & early 00s, including:
 - [OpenZaurus](#) for the [Sharp Zaurus](#)
 - [Familiar Linux](#) for the [iPAQ](#)
 - [OpenSIMpad](#) for the [SIMpad](#)
 - Recipe- and layer-based build system
- [Yocto Project](#) (2010)
 - Initiated by the [Linux Foundation](#) with strong industry/vendor support
 - Tightly coupled/aligned with OpenEmbedded community and tooling
- [BitBake](#)
 - OpenEmbedded's build tool (parses recipes and executes tasks)
 - Official build tool of Yocto Project
- [Poky](#)
 - A reference embedded Linux distribution



Yocto 1 – Core Concepts (“Data Model”)



Yocto 1 – BitBake Language

- Think of BitBake as a glorified set of shell scripts! Except ...
 - ... you can also have Python
 - ... variable expansion is fancier
 - ... variables can be overridden/inherited almost like C++
 - ... most operations are scripted for you, and controlled by variables
- Each recipe has its own variable scope
 - Variables set in a recipe do not affect other recipes
 - Variables set in global configs affect all recipes
- If you understand the BitBake language, and how to find the files, you can discover almost all the functionality of BitBake and Yocto
- (If you understand Python, and where to look, you can discover the rest – but you may never need to go that deep!)

Yocto 1 – Variable Operations

- Variables can be set or modified in several ways:
- Simple set: `FOO = "bar"`
- Default set: `FOO ?= "bar"` (takes effect only if FOO not already set)
- Weak default set: `FOO ??= "bar"` (takes effect only if FOO is *nowhere* set)
- Normally, all variables are lazy-expanded (after all parsing)
- Immediate expand: `FOO := "${BAR}"` (value of BAR taken *now*, not at the end)
- Append/prepend
 - Immediate
 - With spaces: `FOO += "bar"`, `FOO =+ "bar"`
 - Without spaces: `FOO .= "bar"`, `FOO =. "bar"`
 - Lazy
 - (Always without spaces) `FOO:append = "bar"`, `FOO:prepend = "bar"`
- Remove (removes all instances of a whitespace-delimited token)
 - `FOO:remove = "bar"` (always lazy)

Yocto 1 – Variable Overrides

- Overrides conditionally set or modify a variable:
 - `FOO:aarch64 = "bar"`
- Can be combined with modifiers:
 - `FOO:aarch64 += "bar"`
 - `FOO:remove:aarch64 = "bar"`
- You can add anything to the overrides list:
 - `OVERRIDES .= ":foo"`
- The default list includes:
 - `CLASSOVERRIDE ?= "class-target"`
 - `MACHINEOVERRIDES ?= "${MACHINE}"`
 - `DISTROOVERRIDES ?= "${DISTRO}"`

Yocto 1 – Parsing Order

- Note: Some obscure steps are omitted... see link above for full list
- site.conf
- auto.conf
- local.conf
- machine/\${MACHINE}.conf
- distro/\${DISTRO}.conf
- machine config
- distro config
- recipe (*.bb)
- appends (*.bbappend), in layer order

Yocto 1 – Key Variables

- Recipe Metadata
 - DESCRIPTION – description of the package
 - PN – package name
 - BPN – base package name, with suffixes like -native removed
 - PV – package version (can be automatically set from recipe .bb filename)
- Directories
 - WORKDIR – top working directory for recipe: `${TMPDIR}/work/${MULTIMACH_TARGET_SYS}/${PN}/${EXTENDPE}${PV}-${PR}`
 - S – location of unpacked source code: `${WORKDIR}/${BPN}-${PV}`
 - D – “destination” directory where files should be “installed”: `${WORKDIR}/image`
 - B – build directory: `${WORKDIR}/${BP}`
 - DEPLOYDIR – target directory for deploy class: `${WORKDIR}/deploy-${PN}`
 - Do not confuse with DEPLOY_DIR (`${TMPDIR}/deploy`)!
- Source
 - SRC_URI – URIs of source files/archives to be fetched / cloned and extracted
 - SRCREV (for Git repos) – revision (commit hash or tag) to check out
- CMake
 - EXTRA_OECMAKE – extra CMake command line options

Yocto 1 – Tasks – Typical/Core

- **build** – the default task
 - doesn't actually build (or do) anything (don't try!)
 - placeholder that depends on all other tasks
- fetch – downloads source
- unpack – unpacks fetched source (e.g., extract tarball)
- patch – applies any .patch files in SRC_URI
- **configure** – prepares the source for building
- **compile** – compiles package in build directory
- **install** – installs files to a staging directory
- package
- **deploy** (only with inherit deploy)
- **There are many more**, but these are rarely modified

Yocto 1 – Tasks – Adding Custom Tasks

- addtask rub_genie after do_configure before do_compile
- ```
do_rub_genie() {
 # shell script here
}
```
- ```
python do_rub_genie() {  
    # python script here  
}
```
- Tasks have their own dependencies:
 - do_rub_genie[depends] += “foo:do_deploy”
- Task functions (built-in or custom) **can be modified** with append/prepend, overrides, etc., just like variables

Yocto 1 – Packaging

- “Staged” recipes are packaged for later use
 - ... maybe included in rootfs
 - ... maybe exposed to another recipe in its recipe-sysroot (so the other recipe can link to a library, for example)
- Several package formats are supported
 - [.ipk](#), [.rpm](#), [.deb](#)
- For each supported package format, several packages are generated:
 - `${PN}` – contains stuff that should be installed in rootfs (executables, shared libs, resource files)
 - `${PN}-dbg` – contains unstripped (with symbols) variant of the above
 - `${PN}-dev` – contains stuff that another recipe would need: header files, static libraries (goes into recipe-sysroot)
 - `${PN}-src` – contains the source code that was used to build

Yocto 1 – Tools

- See variable values for a recipe: `bitbake -e [-c taskname] recipename`
- List all layers: `bitbake-layers show-layers`
- List recipes: `bitbake-layers show-recipes [-f] [recipename]`
- Graphically explore dependency tree: `bitbake -g -u taskexp [-c taskname] recipename`
- Open shell: `bitbake -c devshell recipename`

Yocto 1 – devtool

- devtool modify: Temporarily hacks layers & appends, so that a package is built from a local source directory instead of fetched URIs (tarballs or Git repositories)
 1. Creates a local workspace layer in build/workspace and adds this layer to bblayers.conf (be careful not to commit this!)
 2. Fetches and extracts SRC_URI into build/workspace/\${PN}
 3. Creates a .bbappend in the build/workspace layer
 - Uses externalsrc.bbclass, just like e.g. FB_LOCAL_PATH
 - Has some issues with e.g. multiple Git repositories in SRC_URI
- devtool update-recipe – Creates patch files for any local modifications and updates the .bb (or makes a .bbappend) to include the patch
 - Isn't always smart about where to put files – especially when “dynamic layers” are involved

Yocto 1 – Digging In & Troubleshooting

- What's installed in the rootfs?
 - image recipes generate a .manifest file in `${DEPLOY_DIR_IMAGE}` (tmp/deploy/images/\${MACHINE})
- What does a package install?
 - Check its package file
 - `rpm -qlp tmp/deploy/rpm/*/*.rpm`
 - `dpkg -c tmp/deploy/ipk/*/*.ipk` (or this workaround if dpkg not installed)
 - Run **bitbake -c install recipename**
 - Look in tmp/work/*/recipename-*/image
- Why is a step failing?
 - Check environment (**bitbake -e recipename**)
 - Try a devshell (**bitbake -c devshell recipename**)

Yocto 1 – Questions?

Thanks for your time!